(

inicio( )

instrucciones( )

functions func main( )

) (block { }

( ) ( )

instrucciones( )

varDcl mut puntos( ) ( )

assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "=== Archivo de prueba básico ==="

instrucciones( )

imprimir println( )

( )

expr "==== Declaración de variables ===="

instrucciones( )

varDcl mut puntos_declaracion( ) ( )

assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "Declaración explícita con tipo y valor"

instrucciones( )

varDcl mut entero( ) ( ) ( )

type int assign = expr 42

instrucciones( )

varDcl mut decimal( ) ( ) ( )

type float64 assign = expr 3.14159

instrucciones( )

varDcl mut texto( ) ( ) ( )

type string assign = expr "Hola, mundo!"

instrucciones( )

varDcl mut booleano( ) ( ) ( )

type bool assign = expr true

instrucciones( )

imprimir println( )

( ) ,( )

expr "entero:" expr entero

instrucciones( )

imprimir println( )

( ) ,( )

expr "decimal:" expr decimal

instrucciones( )

imprimir println( )

( ) ,( )

expr "texto:" expr texto

instrucciones( )

imprimir println( )

( ) ,( )

expr "booleano:" expr booleano

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr entero expr 42

expr( ) >( )

expr decimal expr 3.0

expr( ) ==( )

expr texto expr "Hola, mundo!"

expr( ) ==( )

expr booleano expr true

block { }

( ) ( )

instrucciones( )

varAsign puntos_declaracion +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Declaración explícita con tipo y valor: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Declaración explícita con tipo y valor: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nDeclaración explícita con tipo y sin valor"

instrucciones( )

varDcl mut entero_sin_valor( ) ( )

assign = expr 0

instrucciones( )

varDcl mut decimal_sin_valor( ) ( )

assign = expr 0.0

instrucciones( )

varDcl mut texto_sin_valor( ) ( )

assign = expr ""

instrucciones( )

varDcl mut booleano_sin_valor( ) ( )

assign = expr false

instrucciones( )

imprimir println( )

( ) ,( )

expr "enteroSinValor:" expr entero_sin_valor

instrucciones( )

imprimir println( )

( ) ,( )

expr "decimalSinValor:" expr decimal_sin_valor

instrucciones( )

imprimir println( )

( ) ,( )

expr "textoSinValor:" expr texto_sin_valor

instrucciones( )

imprimir println( )

( ) ,( )

expr "booleanoSinValor:" expr booleano_sin_valor

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr entero_sin_valor expr 0

expr( ) ==( )

expr decimal_sin_valor expr 0.0

expr( ) ==( )

expr texto_sin_valor expr ""

expr( ) ==( )

expr booleano_sin_valor expr false

block { }

( ) ( )

instrucciones( )

varAsign puntos_declaracion +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Declaración explícita con tipo y sin valor: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Declaración explícita con tipo y sin valor: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nErrores de redeclaración"

instrucciones( )

varDcl mut entero( ) ( ) ( )

type int assign = expr 100

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr entero expr 100

block { }

( ) ( )

instrucciones( )

varAsign puntos_declaracion -=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "X Errores de redeclaración: incorrecto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "OK Errores de redeclaración: correcto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Asignación de variables ===="

instrucciones( )

varDcl mut puntos_asignacion( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "Asignación con tipo correcto"

instrucciones( )

varAsign entero =( )

expr 99

instrucciones( )

varAsign decimal =( )

expr 9.9

instrucciones( )

varAsign texto =( )

expr "Texto modificado"

instrucciones( )

varAsign booleano =( )

expr !( )

expr booleano

instrucciones( )

imprimir println( )

( ) ,( )

expr "entero:" expr entero

instrucciones( )

imprimir println( )

( ) ,( )

expr "decimal:" expr decimal

instrucciones( )

imprimir println( )

( ) ,( )

expr "texto:" expr texto

instrucciones( )

imprimir println( )

( ) ,( )

expr "booleano:" expr booleano

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr entero expr 99

expr( ) ==( )

expr decimal expr 9.9

expr( ) ==( )

expr texto expr "Texto modificado"

expr( ) ==( )

expr booleano expr false

block { }

( ) ( )

instrucciones( )

varAsign puntos_asignacion +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Asignación con tipo correcto: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Asignación con tipo correcto: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nAsignación con tipo incorrecto"

instrucciones( )

imprimir println( )

( )

expr "OK Asignación con tipo incorrecto: Se detectaron errores de tipo correctamente"

instrucciones( )

varAsign puntos_asignacion +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "\n==== Operaciones Aritméticas ===="

instrucciones( )

varDcl mut puntos_ari( ) ( ) ( )

type int assign = expr 0

instrucciones( )

varDcl resultado_suma_1( ) ( )

assign := expr( ) +( )

expr 10 expr 5

instrucciones( )

varDcl resultado_suma_2( ) ( )

assign := expr( ) +( )

expr 10.5 expr 5.5

instrucciones( )

varDcl resultado_suma_3( ) ( )

assign := expr( ) +( )

expr 10 expr 5.5

instrucciones( )

varDcl resultado_suma_4( ) ( )

assign := expr( ) +( )

expr 10.5 expr 5

instrucciones( )

imprimir println( )

( ) ,( )

expr "10 + 5 =" expr resultado_suma_1

instrucciones( )

imprimir println( )

( ) ,( )

expr "10.5 + 5.5 =" expr resultado_suma_2

instrucciones( )

imprimir println( )

( ) ,( )

expr "10 + 5.5 =" expr resultado_suma_3

instrucciones( )

imprimir println( )

( ) ,( )

expr "10.5 + 5 =" expr resultado_suma_4

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr resultado_suma_1 expr 15

expr( ) ==( )

expr resultado_suma_2 expr 16.0

expr( ) ==( )

expr resultado_suma_3 expr 15.5

expr( ) ==( )

expr resultado_suma_4 expr 15.5

block { }

( ) ( )

instrucciones( )

varAsign puntos_ari +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Suma: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Suma: incorrecto"

instrucciones( )

varDcl resultado_resta_1( ) ( )

assign := expr( ) -( )

expr 10 expr 5

instrucciones( )

varDcl resultado_resta_2( ) ( )

assign := expr( ) -( )

expr 10.5 expr 5.5

instrucciones( )

varDcl resultado_resta_3( ) ( )

assign := expr( ) -( )

expr 10 expr 5.5

instrucciones( )

varDcl resultado_resta_4( ) ( )

assign := expr( ) -( )

expr 10.5 expr 5

instrucciones( )

imprimir println( )

( ) ,( )

expr "10 - 5 =" expr resultado_resta_1

instrucciones( )

imprimir println( )

( ) ,( )

expr "10.5 - 5.5 =" expr resultado_resta_2

instrucciones( )

imprimir println( )

( ) ,( )

expr "10 - 5.5 =" expr resultado_resta_3

instrucciones( )

imprimir println( )

( ) ,( )

expr "10.5 - 5 =" expr resultado_resta_4

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr resultado_resta_1 expr 5

expr( ) ==( )

expr resultado_resta_2 expr 5.0

expr( ) ==( )

expr resultado_resta_3 expr 4.5

expr( ) ==( )

expr resultado_resta_4 expr 5.5

block { }

( ) ( )

instrucciones( )

varAsign puntos_ari +=( )

expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Resta: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Resta: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Operaciones Relacionales ===="

instrucciones( )

varDcl mut puntos_rel( ) ( )

assign = expr 0

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) ==( )

expr 10 expr 10

expr( ) !=( )

expr 10 expr 5

expr( ) ==( )

expr 10.5 expr 10.5

expr( ) !=( )

expr 10.5 expr 5.5

block { }

( ) ( )

instrucciones( )

varAsign puntos_rel +=( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "OK Relacionales: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Relacionales: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Operaciones Lógicas ===="

instrucciones( )

varDcl mut puntos_log( ) ( )

assign = expr 0

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr( ) &&( )

expr true expr true

expr !( )

expr( )

( )

expr( ) &&( )

expr false expr true

expr( )

( )

expr( ) &&( )

expr( ) ==( )

expr 10 expr 10

expr( ) ==( )

expr 5 expr 5

expr( )

( )

expr( ) ||( )

expr true expr false

expr !( )

expr( )

( )

expr( ) ||( )

expr false expr false

block { }

( ) ( )

instrucciones( )

varAsign puntos_log +=( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "OK Lógicas: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Lógicas: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== println ===="

instrucciones( )

varDcl mut puntos_print( ) ( )

assign = expr 2

instrucciones( )

imprimir println( )

( )

expr 42

instrucciones( )

imprimir println( )

( )

expr 3.14

instrucciones( )

imprimir println( )

( )

expr "Texto de prueba"

instrucciones( )

imprimir println( )

( )

expr true

instrucciones( )

imprimir println( )

( ) ,( ) ,( )

expr "Hola" expr 123 expr true

instrucciones( )

imprimir println( )

( )

expr "\n==== Manejo de valor nulo ===="

instrucciones( )

varDcl mut puntos_nil( ) ( )

assign = expr 2

instrucciones( )

imprimir println( )

( )

expr "\n==== Punto y coma opcional ===="

instrucciones( )

varDcl mut puntos_semicolon( ) ( )

assign = expr 2

instrucciones( )

varDcl a( ) ( )

assign := expr 10

instrucciones( )

varDcl b( ) ( )

assign := expr 20

instrucciones( )

varDcl c( ) ( )

assign := expr( ) +( )

expr a expr b

instrucciones( )

imprimir println( )

( ) ,( )

expr "a =" expr a

instrucciones( )

imprimir println( )

( ) ,( )

expr "b =" expr b

instrucciones( )

imprimir println( )

( ) ,( )

expr "c = a + b =" expr c

instrucciones( )

varAsign puntos =( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr( ) +( )

expr puntos_declaracion expr puntos_asignacion

expr puntos_ari

expr puntos_rel

expr puntos_log

expr puntos_print

expr puntos_nil

expr puntos_semicolon

instrucciones( )

imprimir println( )

( )

expr "\n=== Tabla de Resultados ==="

instrucciones( )

imprimir println( )

( )

expr "| Asignación de variables"

instrucciones( )

imprimir println( )

( )

expr puntos_asignacion

instrucciones( )

imprimir println( )

( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "| Operaciones Aritméticas |"

instrucciones( )

imprimir println( )

( )

expr puntos_ari

instrucciones( )

imprimir println( )

( )

expr 2

instrucciones( )

imprimir println( )

( )

expr "| Operaciones Relacionales |"

instrucciones( )

imprimir println( )

( )

expr puntos_rel

instrucciones( )

imprimir println( )

( )

expr "| Operaciones Lógicas |"

instrucciones( )

imprimir println( )

( )

expr puntos_log

instrucciones( )

imprimir println( )

( )

expr "| println | "

instrucciones( )

imprimir println( )

( )

expr puntos_print

instrucciones( )

imprimir println( )

( )

expr "| Manejo de nulo | "

instrucciones( )

imprimir println( )

( )

expr puntos_nil

instrucciones( )

imprimir println( )

( )

expr "| Punto y coma opcional |"

instrucciones( )

imprimir println( )

( )

expr puntos_semicolon

instrucciones( )

imprimir println( )

( )

expr "| TOTAL |"

instrucciones( )

imprimir println( )

( )

expr puntos

)